

Leva.Framework.Sample.StateCounter

Web sample for Core, Engine, Execution, and Fakes. It shows the reusable workflow setup in one small app: states consume events, a behavior handles a delayed evaluation event, AlarmBoard and StatusBoard capture runtime facts, and ExecutionRunner creates a report.

```
Leva.Framework.Sample.StateCounter
```

- > Leva.Framework.Core
- > Leva.Framework.Engine
- > Leva.Framework.Execution
- > Leva.Framework.Fakes
- > Microsoft.AspNetCore

Setup shown by the sample

```
Program.cs
```

- > serves the web page
- > maps POST /workflow/run
- > calls StateCounterRunner.RunAsync()

```
StateCounterRunner
```

- > creates FakeClock and FakeLogSink
- > creates ExecutionBoard and ExecutionRunner
- > builds Context with ContextBuilder
- > registers IdleState, CountingState, CompletedState
- > registers TargetReachedBehavior
- > queues StartCounterEvent and IncrementCounterEvent
- > schedules EvaluateCounterEvent with EnqueueDelayedAsync()
- > runs EventLoop.RunOneAsync() for visible sample execution

```
CounterAccess
```

- > extends IAccess with CounterModel and ExecutionRunner
- > exposes Context-owned AlarmBoard and StatusBoard without giving states the whole Context

Behavior shown

```
StartCounterEvent      -> IdleState transitions to Counting
IncrementCounterEvent  -> CountingState increments the model
EvaluateCounterEvent   -> TargetReachedBehavior updates status, raises warning alarm, transitions to Completed
CompletedState         -> ExecutionRunner runs CounterReportExecution
```

Run and expected result

```
dotnet run --project samples/Leva.Framework.Sample.StateCounter
```

```
Sample result: PASS
State: Completed
Count: 2
Alarms: 1 warning alarm
Statuses: Target count = 2
Execution entries: 1 completed report execution
```

The runtime log is displayed with simplified entries so the page shows framework behavior without exposing compiler-generated method names.